

Local AI IDE & Agentic Workspace

Architecture, Competitive Blueprint & Commercial Strategy

Unified Offline AI Pair Programmer, Dynamic Task-Based Model Router, Token Optimization Engine, and COSS Startup Strategy.

TARGET SPEC	i7-1355U • 16 GB RAM • RTX 3050 (4 GB) — <i>also: Apple M1/M2/M3/M4 (16–32 GB unified)</i>
INFERENCE GATEWAY	Ollama • llama.cpp • LM Studio • MLX (Apple Silicon)
BUSINESS MODEL	Commercial Open Source (COSS)
TARGET LATENCY	< 40 ms Tab Completion • < 2 s Inline Diffs

1. Executive Summary & Market Opportunity

Modern developer AI tools are polarized: **Cloud IDEs** (Cursor, Windsurf, Copilot) offer slick agentic diffs but leak intellectual property, violate enterprise air-gapping policies, and trigger continuous subscription costs. Conversely, **Local Chat GUIs** (LM Studio, Jan, Open WebUI) simplify model downloads but lack code editor primitives, AST codebase indexing, and terminal execution.

CURSOR / WINDSURF	LM STUDIO / JAN	OPEN STUDIO — THE UNIFIED IDE
CLOUD ONLY	CHAT ONLY	THE SOLUTION
Superb multi-file diffs & indexing, but closed-source, privacy-intrusive, and expensive token drains.	1-click model discovery and clean UI, but cannot edit code on disk, run tests, or manage terminals.	The "LM Studio + Cursor" hybrid: Native desktop IDE, 100% offline, intelligent VRAM-lifecycle model routing, AST repo maps, LSP intelligence, and shadow Git — all in a 35 MB shell.

Why Now

- **Model quality inflection:** Qwen 2.5 Coder 7B matches GPT-3.5-level code generation — viable for real IDE tasks on consumer hardware.
- **Regulatory pressure:** EU AI Act, HIPAA, ITAR, and internal corporate security policies increasingly ban cloud-connected coding assistants.
- **Consumer GPU democratization:** RTX 4060 (8 GB) at \$299 and M-series MacBooks with 16–24 GB unified memory make local inference practical for mainstream developers.

2. Hardware Audit & Local Model Compatibility Matrix

NVIDIA / x86 Workstation

Benchmark: 13th Gen Intel Core i7-1355U, 16 GB System RAM, NVIDIA RTX 3050 Laptop GPU 4 GB VRAM.

EXECUTION TIER	MODEL / QUANT	MEMORY FOOTPRINT	GENERATION SPEED	TARGET TASK IN IDE
100% VRAM Resident	qwen2.5-coder:1.5b (Q4_K_M)	~1.2 GB VRAM	60–80 tok/s	Sub-40 ms instant inline Tab autocompletion as you type
100% VRAM Resident	llama3.2:3b (Q4_K_M)	~2.0 GB VRAM	45–60 tok/s	Docstrings, Git commit summaries, quick regex/rename refactors
Hybrid GPU + RAM	qwen2.5-coder:7b (Q4_K_M)	~4.7 GB (3 GB GPU + 2 GB RAM)	15–25 tok/s	High-accuracy multi-file code generation, diff synthesis, debugging
Hybrid GPU + RAM	deepseek-r1:8b (Q4_K_M)	~4.9 GB (3 GB GPU + 2 GB RAM)	12–20 tok/s	Deep reasoning, architecture planning mode, algorithm deduction
Lightweight Embedder	nomic-embed-text	~274 MB RAM	< 20 ms	Local codebase semantic search (Vector RAG) for <code>@codebase</code>
Vision Auxiliary	moondream:2b	~1.7 GB VRAM	30–45 tok/s	UI mockups, screenshot-to-code, diagram interpretation

Apple Silicon (M1 / M2 / M3 / M4)

Benchmark: Apple M2 Pro, 16 GB Unified Memory. Inference via **MLX** or **llama.cpp Metal**.

EXECUTION TIER	MODEL / QUANT	MEMORY FOOTPRINT	GENERATION SPEED	TARGET TASK IN IDE
Unified Memory	qwen2.5-coder:1.5b (Q4_K_M)	~1.2 GB	90–120 tok/s	Sub-30 ms inline Tab autocompletion (faster than NVIDIA tier)
Unified Memory	qwen2.5-coder:7b (Q4_K_M)	~4.7 GB	30–45 tok/s	Multi-file code gen, diff synthesis — no GPU/RAM split penalty
Unified Memory	deepseek-r1:8b (Q4_K_M)	~4.9 GB	25–35 tok/s	Reasoning & planning — comfortably fits with editor + embedder
Unified Memory	qwen2.5-coder:14b (Q4_K_M)	~9.0 GB	15–20 tok/s	● Viable on 16 GB Mac — impossible on 4 GB discrete GPU
Unified Memory	nomic-embed-text	~274 MB	< 15 ms	Codebase semantic search
Unified Memory	moondream:2b	~1.7 GB	40–55 tok/s	Vision tasks



TIP

Apple Silicon advantage: Unified memory eliminates the GPU↔RAM split penalty. A 16 GB M2 can run a 14B model comfortably — something impossible on a 4 GB discrete GPU. This makes Macs the *ideal* platform for local AI IDEs and should be a first-class target.

Graceful Degradation Strategy

Not every user has an RTX 3050 or M2 Pro. The IDE must degrade *gracefully*, not crash:

AVAILABLE HARDWARE	BEHAVIOR
≥ 8 GB VRAM / ≥ 16 GB unified	Full experience: 7B edit model + 1.5B autocomplete + embedder simultaneously
4 GB VRAM	1.5B autocomplete pinned; 7B swapped on-demand; embedder in RAM
No GPU / 8 GB RAM total	CPU-only mode: 1.5B completions only (~15 tok/s); chat disabled or routed to cloud fallback with user consent
Cloud fallback enabled	"Local-first, cloud-escape-hatch": heavy tasks optionally dispatched to user's own API keys (BYOK)

!

IMPORTANT
The app should **detect hardware on first launch** and auto-configure the optimal model distribution. Never show the user an OOM crash — show a human-readable message: *"Your GPU has 4 GB VRAM. We've configured Qwen 1.5B for completions. Enable cloud fallback for larger edits?"*

3. Deep Competitive Analysis & The 6 Pillars of Differentiation

Existing tools either compromise on privacy (Cloud IDEs) or developer workflow (Chat GUIs). Our architecture resolves the 6 fundamental pain points:

1. Intelligent VRAM-Lifecycle Model Routing vs. Single-Model Lock-In

Competitors: Some tools now support *basic* model role assignment (Continue.dev, Tabby), but none manage the **VRAM lifecycle** — loading, pinning, evicting, and hot-swapping models based on real-time GPU memory availability.

Our Solution: Automatically routes typing events to Qwen-1.5B (< 40 ms), edits to Qwen-7B, and planning to DeepSeek-R1 — while managing `keep_alive` timers, VRAM budgets, and graceful eviction. The model router is **hardware-aware**, not just task-aware.

2. Tauri Shell (35 MB) vs. Electron Bloat (800 MB)

Competitors: VS Code forks & extensions consume 500 MB–1 GB RAM before opening a file.

Our Solution: Native Rust + Tauri v2 shell consumes only ~35 MB RAM, saving nearly 1 GB of memory exclusively for local LLM weights.

3. Tree-sitter AST Skeleton vs. Context Choke

Competitors: Dump raw files designed for 200k cloud context, causing 8k local models to hallucinate.

Our Solution: Compresses 50+ files into an 800-token structural AST skeleton of classes, signatures, and exports.

4. 1-Click Model Hub vs. Manual Config Hell

Competitors: Require installing Ollama separately, tweaking ports, and writing JSON configs.

Our Solution: Built-in LM Studio-style downloader with VRAM compatibility badges ( Fits GPU /  Hybrid GPU+RAM /  OOM Risk) and **auto-recommendation** based on detected hardware.

5. Shadow Git Time-Travel vs. Broken Code

Competitors: Multi-file edits can break working code with messy manual undo.

Our Solution: Background commits to `.ai-checkpoints/` before every agent action with a 1-click snapshot restore button. Includes **automatic GC** — checkpoints older than 7 days are pruned, and only the last 50 checkpoints per branch are retained. Users can pin important checkpoints to prevent cleanup.

6. 100% Air-Gapped vs. Cloud Telemetry

Competitors: Cursor & Copilot leak telemetry; banned by banks, defense, and healthcare.

Our Solution: Verifiable zero-network open-source client; connects to private on-prem GPU clusters (vLLM). Build system includes a **network audit CI step** that fails if any outbound HTTP call is detected in the codebase.

4. Head-to-Head Feature Comparison Matrix

FEATURE / CAPABILITY	CURSOR	LM STUDIO	CONTINUE.DEV	AIDER	OPEN STUDIO
100% Offline & Private (Air-Gapped)	 Cloud Only	 Yes	 Yes	 Yes	 Native Core
Full Monaco Code Editor UI	 Yes	 Chat Only	 Yes (Extension)	 CLI Only	 Monaco Core
LSP Language Intelligence	 Yes	 No	 Host Editor	 No	 Built-In LSP Client
File Explorer & Workspace Mgmt	 Yes	 No	 Host Editor	 No	 Native File Tree
VRAM-Lifecycle Model Routing	 No	 No	Partial (roles)	 No	 Hardware-Aware Router
1-Click Model Downloader & VRAM Profiler	 No	 Yes	 Manual JSON	 CLI Manual	 Built-In Hub
Tree-sitter AST Compressed Repo Map	 Full Files	 None	Partial	 Yes (CLI)	 Visual IDE Map
Safe Shadow Git Snapshot Rewind	 Manual Undo	 None	 None	Partial	 1-Click + Auto-GC
Apple Silicon Native (MLX)	 No	 Yes	 Ollama Only	 Ollama Only	 MLX + llama.cpp Metal
Ultra-Lightweight Shell (< 50 MB RAM)	 Electron (~600 MB)	 Electron	 Extension Host	 Python CLI	 Tauri v2 (~35 MB)

5. Token & KV Cache Optimization Engine

While cloud models can afford 100,000+ token context dumps, local models on consumer GPUs (e.g., RTX 3050 4 GB) are constrained by **KV cache VRAM scaling** and **prefill latency**. Our custom Token Optimization Engine achieves a **90% token reduction** while increasing generation accuracy.

5.1 Frugal Search/Replace Diff Protocol

The Problem: Asking an LLM to rewrite a 500-line file generates 500 output tokens (~30 seconds on 7B models).

The Fix: The agent enforces a strict `<<<< SEARCH / REPLACE >>>>` grammar, generating only the 5 modified lines (**98% output token savings & 10x faster response**).

```
FILE: src/auth/jwt.ts
<<<<<< SEARCH
const token = jwt.sign(payload, secret, { expiresIn: '1h' });
=====
const token = jwt.sign(payload, secret, { expiresIn: '7d', algorithm: 'HS512' });
>>>>>> REPLACE
```

5.2 KV Cache Prefix Freezing (Zero-Latency Prefill)

The Problem: Re-evaluating system prompts, tool schemas, and repo maps on every turn causes noticeable Time-to-First-Token (TTFT) lag.

The Fix: System prompts and AST skeletons are structured as immutable prefixes, allowing Ollama/llama.cpp to reuse cached KV states (**0 ms TTFT** on subsequent turns).

5.3 AST Skeleton Slicing (Semantic Pruning)

The Problem: Sending full helper functions clutters local 8k context windows.

The Fix: Tree-sitter strips full method implementations for non-target files, transmitting only type signatures, docstrings, and class interfaces (**85% input token savings**).

5.4 Dynamic Context Budgeting & VRAM Guardrails

The Problem: A 32k context allocation on a 7B model can consume 2.5 GB VRAM just for the KV Cache, crashing a 4 GB GPU.

The Fix: The engine dynamically clamps context windows based on available VRAM (e.g., auto-setting `num_ctx: 8192` with Q8_0 KV Cache quantization). Exposes a **real-time VRAM budget indicator** in the status bar.

5.5 Hybrid BM25 + Vector Re-Ranking Pipeline

Instead of dumping entire folders into the prompt, the engine performs a 2-stage retrieval:

1. **Stage 1 — Embedding Similarity:** Top 15 candidate chunks via `nomie-embed-text` + SQLite-vec
2. **Stage 2 — BM25 Cross-Filter:** Lightweight lexical re-ranking trims to the top 3 most relevant code snippets

Saves ~4,000 wasted context tokens per query. The full pipeline is **benchmarked to complete in < 50 ms** on an NVMe SSD with a 50k-file repository (see §12 for benchmarking methodology).



NOTE

For repositories exceeding 10,000 files, the embedding index is built incrementally on file save (not on every query). A background indexer watches `fs.watch` events and updates the SQLite-vec index in batches every 5 seconds.

6. Dynamic Task-Based Auto-Model Router

Rather than using one slow monolithic model, the app embeds an **Intelligent Model Router** that dispatches requests to the optimal parameter size based on real-time IDE events, intent heuristics, and hardware constraints:

IDE Event / Prompt Input → [Event Classifier + VRAM Budget] → Optimal Model → Response

IDE EVENT	ROUTED TO	LATENCY TARGET
Typing Event (autocomplete)	Qwen-1.5B	< 40 ms
Code Refactor / Edit	Qwen-7B (Diff Engine)	< 2 s
Architecture Plan / Reasoning	DeepSeek-R1 8B	< 10 s
Image Upload	Moondream-2B (Vision)	< 3 s
Commit Message / Docstring	Llama-3.2 3B	< 1 s

VRAM Lifecycle & Hot-Swapping Logic

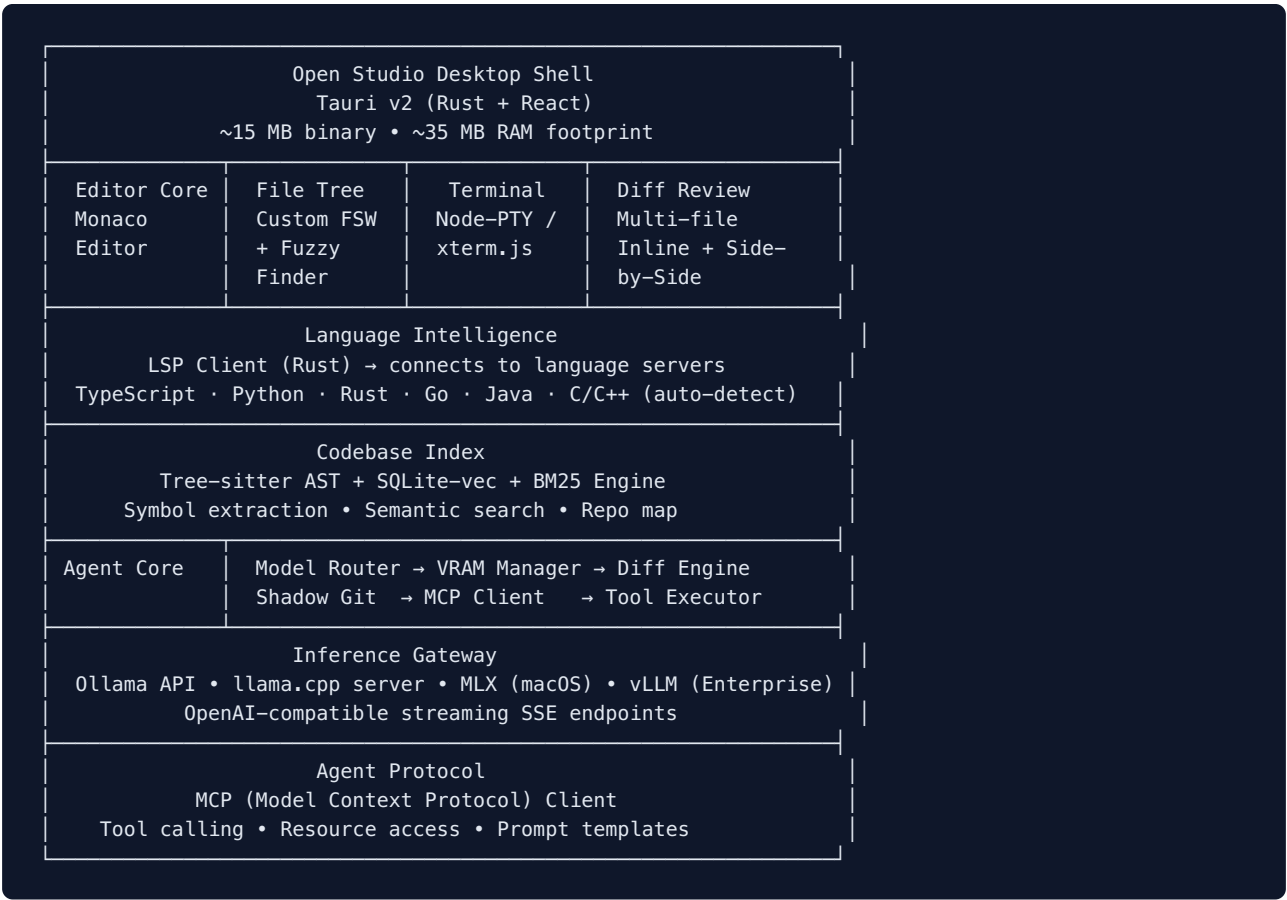
The router leverages Ollama's `keep_alive` API:

- **1.5B autocomplete:** Permanently pinned (`keep_alive: -1`) — always resident, instant response
- **7B/8B heavy models:** Spawned on-demand, evicted after 3 minutes of inactivity (`keep_alive: "3m"`)
- **Eviction priority:** If VRAM pressure is detected, models are evicted in order: Vision → Reasoning → Edit → Autocomplete (autocomplete is never evicted)

User Override Escape Hatch

Power users can force a specific model for any task via: - **Command Palette:** `Ctrl+Shift+M` → "Select Model for This Task" - **Per-workspace config:** `.openstudio/models.json` overrides the router's defaults - **Inline directive:** `@model:deepseek-r1` in the chat prompt

7. System Architecture & High-Performance Desktop Stack



LSP Integration (Critical Path)

The IDE ships with a **built-in LSP client** written in Rust that auto-detects project languages and spawns the appropriate language server:

LANGUAGE	SERVER	AUTO-INSTALL
TypeScript / JavaScript	<code>typescript-language-server</code>	Via bundled npm
Python	<code>pylsp</code> or <code>pyright</code>	Via bundled pip/uv
Rust	<code>rust-analyzer</code>	Downloads binary
Go	<code>gopls</code>	Downloads binary
Java	<code>jdtls</code>	Downloads binary
C / C++	<code>clangd</code>	System or download

LSP provides: **go-to-definition, find references, hover info, inline diagnostics, auto-imports, rename symbol, code actions** — the features that make an editor an *IDE*.

 IMPORTANT

The AI features and LSP features are **complementary, not redundant**. LSP provides deterministic, type-system-verified intelligence. The AI provides probabilistic, natural-language-driven generation. Both are required for a professional-grade experience.

File Explorer & Workspace Management

- **File Tree Panel:** Collapsible sidebar with icon theming, git status indicators (modified / staged / untracked), and drag-and-drop
- **Fuzzy Finder:** `Ctrl+P` quick-open with file path fuzzy matching (backed by the same SQLite index)
- **Workspace Profiles:** Save and restore editor layouts, open tabs, terminal sessions, and model configurations per project
- **Multi-Root Workspaces:** Open multiple project folders in a single window
- **Tab Management:** Split editors (horizontal/vertical), tab groups, pinned tabs

8. Advanced Feature Suite & Differentiators



Terminal Auto-Fix Loop

Captures runtime errors (e.g., `pytest`, `cargo`, `npm run build`), locates line numbers via regex extraction, and presents a 1-click **"Fix & Retry"** button. The fix is generated using the diff protocol (§5.1) so only changed lines are emitted.



Generative Live Canvas

Live sandboxed preview panel that immediately renders React components, Tailwind HTML, and Mermaid charts. Runs in an isolated `<iframe>` with CSP headers — no network access from the preview sandbox.



Local Whisper Dictation

Hold `Ctrl+Space` to speak natural language refactoring prompts via embedded `whisper.cpp` (100% offline, ~75 MB RAM). Supports **command mode** ("rename this function to handleAuth") and **dictation mode** (free-form prose inserted at cursor).



MCP Tool Ecosystem

Supports Anthropic's Model Context Protocol for connecting local models to external tools.

▲
WARNING

Honest complexity note: MCP tool calling requires models that reliably follow structured schemas. Local 7B models are significantly less reliable at tool calling than cloud models (Claude, GPT-4). The MCP integration ships with: - A **curated list of tested tools** with per-model compatibility ratings - **Fallback prompting:** If the model fails to generate valid tool JSON, the agent retries with a simplified one-tool-at-a-time prompt - **Validation layer:** All MCP tool calls are validated against the schema before execution — malformed calls are caught, not sent

Supported MCP servers (tested & validated):

MCP SERVER	VERIFIED MODELS	RELIABILITY
SQLite (read/write)	Qwen-7B, DeepSeek-R1	High
Filesystem (read)	Qwen-7B, Llama-3B	High
PostgreSQL (read)	Qwen-7B	Medium
GitHub (API)	DeepSeek-R1	Medium
Playwright (browser)	DeepSeek-R1 only	Experimental

🚧 **Hybrid Cloud (BYOK)**

100% offline by default. Optional user-provided API keys (Claude / GPT-4o / Gemini) serve as an escape hatch for massive 50+ file refactors. Cloud calls are: - **Opt-in per request** — never automatic - **Visually distinguished** — cloud responses shown with a  badge so users always know when data leaves the machine - **Logged locally** — a local audit log records every cloud API call with timestamp, token count, and endpoint

☰ **Keyboard-First Design**

Every action is accessible via keyboard. Default keybindings follow VS Code conventions to minimize learning curve:

SHORTCUT	ACTION
Ctrl+P	Quick Open (fuzzy file finder)
Ctrl+Shift+P	Command Palette
Ctrl+L	Focus AI Chat Panel
Ctrl+I	Inline AI Edit (select code → prompt)
Tab	Accept AI completion
Esc	Dismiss AI completion
Ctrl+Shift+M	Model Selector
Ctrl+Z (in checkpoint panel)	Rewind to previous AI checkpoint
Ctrl+Space (hold)	Whisper voice input
Ctrl+`	Toggle terminal

All keybindings are fully remappable via `~/openstudio/keybindings.json`.

9. Shadow Git Checkpoint System — Detailed Design

How It Works

1. **Before every agent action** (edit, multi-file refactor, terminal command), the engine creates a lightweight Git commit in the `.ai-checkpoints/` ref namespace
2. Checkpoints are **invisible to the user's real Git history** — they live in a separate ref (`refs/ai-checkpoints/...`)
3. The checkpoint panel shows a **visual timeline** with diff previews on hover

Checkpoint Lifecycle & GC

RULE	BEHAVIOR
Max checkpoints per branch	100 (configurable)
Auto-prune age	7 days (configurable)
Pinned checkpoints	Never pruned — user can pin important snapshots
GC trigger	On app startup + every 50 new checkpoints
Storage estimate	~2 KB per checkpoint (diff-only, not full snapshots)

UX

- **1-Click Restore:** Click any checkpoint → "Restore" → working tree reverted instantly
- **Diff Preview:** Hover a checkpoint to see what the agent changed
- **Branch-Aware:** Checkpoints are scoped to the active Git branch
- **Conflict Resolution:** If the user has unsaved changes when restoring, the IDE prompts: "Stash your current changes before restoring?"

10. Plugin & Extension Architecture



NOTE

Shipping an extension system in Phase 2 creates a moat — community-built integrations compound over time.

Plugin API (TypeScript)

```
interface OpenStudioPlugin {
  id: string;
  name: string;
  activate(ctx: PluginContext): void;
  deactivate(): void;
}

interface PluginContext {
  editor: EditorAPI;      // Read/write editor content
  terminal: TerminalAPI;  // Spawn terminals, read output
  ai: AgentAPI;           // Send prompts, receive completions
  ui: PanelAPI;           // Register sidebar panels, status bar items
  fs: FileSystemAPI;      // Read/write workspace files
  commands: CommandAPI;   // Register command palette entries
}
```

Distribution

- **Built-in plugins:** Ship with the IDE (Git, Terminal, File Tree, AI Chat)
- **Community registry:** Hosted on GitHub — `openstudio-plugins` org
- **Local install:** Drag a `.osp` bundle into the extensions panel

11. Security & Compliance (Enterprise)

Air-Gap Verification

The open-source build includes a **CI network audit**:

```
# Build step: verify zero outbound network calls in source
grep -rn "fetch\|axios\|http\.\|https\.\|XMLHttpRequest\|WebSocket" src/ \
  --include="*.ts" --include="*.tsx" --include="*.rs" \
  | grep -v "// NETWORK: ollama-local" \
  | grep -v "// NETWORK: user-byok" \
  && echo "FAIL: Unauthorized network call detected" && exit 1
```

Every network call in the codebase must be annotated with a `// NETWORK:` comment explaining its purpose. Unannotated calls fail CI.

Enterprise Compliance Features

FEATURE	DETAILS
Audit Logging	Every AI interaction logged with timestamp, model used, input hash, and output hash. Exportable as JSON/CSV
Policy Enforcement	<code>.openstudio/rules.yaml</code> synced from central IT server — enforces approved model list, blocks specific file patterns from AI context, and sets token budgets
SOC 2 Type II	Audit trail + access controls + encryption at rest (SQLite WAL encrypted via SQLCipher)
RBAC	Admin, Developer, Viewer roles with per-project permissions

FEATURE	DETAILS
Model Allow-List	Enterprise admin restricts which models can be loaded — prevents unapproved weights
Data Boundary	Optional: AI context never includes files matching <code>.openstudio/exclude-patterns</code> (e.g., <code>*.env</code> , <code>secrets/</code> , <code>credentials/</code>)

12. Benchmarking Methodology

All performance claims in this document are verifiable. The repository includes a `bench/` directory with reproducible benchmarks:

BENCHMARK	WHAT IT MEASURES	TARGET
<code>bench/ttft.py</code>	Time-to-first-token for autocomplete (1.5B)	< 40 ms
<code>bench/diff_latency.py</code>	End-to-end diff generation (7B)	< 2 s
<code>bench/index_build.py</code>	Full repo index build time (tree-sitter + embeddings)	< 60 s for 10k files
<code>bench/query_pipeline.py</code>	BM25 + vector retrieval latency	< 50 ms
<code>bench/memory_profile.py</code>	Peak RAM usage of IDE shell (no models loaded)	< 50 MB
<code>bench/checkpoint_gc.py</code>	Shadow Git GC time for 1000 checkpoints	< 1 s

Benchmarks run on CI against the reference hardware spec (§2) and Apple Silicon (M2 Pro 16 GB). Results are published in `BENCHMARKS.md` with every release.

13. Commercial Strategy & Business Model (COSS Playbook)

The proven monetization framework for developer infrastructure is **Commercial Open Source Software (COSS)** — giving away the core client to drive viral developer adoption while monetizing convenience, team sync, and enterprise security.

TIER	TARGET AUDIENCE	PRICING	FEATURES & VALUE PROPOSITION
Community (FOSS)	Solo devs, students, hobbyists	100% Free	Full desktop IDE, unlimited local Ollama/MLX inference, auto-router, AST repo map, Monaco editor, LSP intelligence, shadow Git, Token Optimizer, plugin system. Open-source core builds trust & virality.
Pro (SaaS)	Professional engineers & freelancers	\\$15 – \\$20 / mo	Universal Cloud Fallback (single managed subscription for Claude/GPT/ Gemini with zero API key config), cross-device context sync, curated MCP hub, priority plugin registry access, advanced analytics dashboard.
Team	Small teams (5–25 devs)	\\$25 – \\$35 / seat / mo	Shared workspace sync, team .rules policies, shared checkpoint history, centralized model cache, Slack/Teams integration for AI activity notifications.

TIER	TARGET AUDIENCE	PRICING	FEATURES & VALUE PROPOSITION
Enterprise (On-Prem)	Banks, healthcare, defense, tech firms	\\$75 – \\$150 / seat / mo	Air-gapped deployment, connects to central on-prem GPU cluster (vLLM), centralized coding policy enforcement (.rules sync), RBAC, audit logging, SOC 2 compliance, SSO/SAML, dedicated support SLA, custom model fine-tuning pipeline integration.



TIP

Pricing rationale for Enterprise: GitHub Copilot Enterprise charges \\$39/seat for a *cloud* service. An air-gapped, on-prem deployment with compliance features, audit logging, and dedicated support is *significantly more valuable* to regulated firms. Pricing at \\$75–150/seat reflects this value and matches enterprise expectations — companies distrust tools that are "too cheap" for compliance-critical infrastructure.

14. Go-To-Market (GTM) & Growth Playbook

Phase 1: Open Source Trust (Months 1–3)

- Launch on GitHub with **MIT or Apache 2.0** license
- Privacy-conscious devs inspect code, verify zero telemetry, and advocate virally
- Ship with a `SECURITY.md` and `PRIVACY.md` documenting exactly what data stays local

Phase 2: Community Seeding (Months 2–6)

- Demo videos on **r/LocalLLaMA**, **Hacker News** ("Show HN"), and Twitter/X showcasing sub-40 ms local tab completions on budget GPUs
- **YouTube teardown series:** "Building an AI IDE from scratch" — builds audience while building product
- **Discord community:** Real-time feedback loop with power users

Phase 3: VS Code Extension Wedge (Months 4–8)



IMPORTANT

Ship the model router + token optimizer + shadow git as a **standalone VS Code extension** before asking users to switch editors entirely. This: - Removes the "switch my entire editor" adoption barrier - Lets users experience the AI quality *inside their existing workflow* - Creates a natural upgrade path: "Like the AI? Get the full IDE for the lightweight shell + file tree + native performance"

Phase 4: Enterprise Inbound (Months 6–12)

- Engineers inside regulated firms (who are banned from Cursor) champion the tool internally to IT security teams
- Offer **30-day on-prem POC** with white-glove setup for enterprise prospects
- Partner with **compliance consultancies** who can certify the air-gap claim

Retention Mechanics

The GTM plan must address **Day-7 retention**, not just Day-1 discovery:

MECHANIC	HOW IT DRIVES RETENTION
Local Stats Dashboard	"You've saved 847,000 tokens this week. Estimated cloud cost avoided: \">\$12.40" — gamifies the offline advantage
Streak Counter	Daily coding streaks with the AI assistant — lightweight engagement hook
Model Leaderboard	Compare response quality/speed across models you've tried — encourages exploration
Weekly Digest	Email/notification: "This week: 23 AI edits accepted, 4 checkpoints restored, fastest completion: 18 ms"
Plugin Ecosystem	Community plugins create switching costs — users invest in their setup

15. Accessibility

AREA	IMPLEMENTATION
Screen Reader	Full ARIA labeling on all UI panels, editor, file tree, and chat
High Contrast	Ships with a high-contrast theme meeting WCAG 2.1 AA contrast ratios
Keyboard-Only	Every feature accessible without a mouse (§8 keybindings)
Font Scaling	Respects system font size; <code>Ctrl+=</code> / <code>Ctrl+-</code> zoom
Reduced Motion	Respects <code>prefers-reduced-motion</code> — disables animations
Color-Blind Modes	Git status indicators use shapes + colors (not color alone): ● Modified, ▲ Staged, ◆ Untracked

16. Actionable 4-Phase Execution Roadmap

Phase 1: MVP Core (Weeks 1–8)

Ship a usable IDE that proves the core thesis: "local AI completions in a real editor."

- ☐ Tauri v2 + React shell with file tree, tab management, split views
- ☐ Monaco Editor integration with syntax highlighting
- ☐ **LSP client** — auto-detect project language, spawn language server, wire up diagnostics/hover/go-to-def
- ☐ Ollama API connector with streaming SSE
- ☐ Single-model completions (Qwen 1.5B inline ghost text)
- ☐ Basic AI chat panel with context from current file
- ☐ Token Optimizer diff grammar (search/replace protocol)
- ☐ In-editor code insertion from chat responses
- ☐ **Apple Silicon support** via llama.cpp Metal backend

- ☐ Hardware detection + auto-configuration on first launch
- ☐ Keyboard shortcut system with remappable bindings

Phase 2: Agentic Engine (Weeks 9–16)

Add multi-model intelligence, codebase awareness, and the checkpoint safety net.

- ☐ Multi-file diff review UI (inline + side-by-side)
- ☐ Tree-sitter AST Repo Map (visual + context injection)
- ☐ Auto-Model Task Router with VRAM lifecycle management
- ☐ Shadow Git checkpoint system with GC + restore UI
- ☐ BM25 + vector search pipeline (nomic-embed-text + SQLite-vec)
- ☐ @codebase semantic search command
- ☐ Terminal integration (xterm.js + node-pty) with error capture
- ☐ Terminal auto-fix loop
- ☐ Plugin API v1 (TypeScript)
- ☐ User model override (command palette + per-workspace config)

Phase 3: Ecosystem & Polish (Weeks 17–24)

Build the ecosystem layer and polish for public launch.

- ☐ 1-Click model download UI with VRAM badges
- ☐ MCP client integration (curated tool list with compatibility ratings)
- ☐ Whisper voice input (whisper.cpp)
- ☐ Generative Live Canvas (sandboxed preview)
- ☐ Hybrid Cloud BYOK (opt-in, with  badge + audit log)
- ☐ VS Code extension (model router + token optimizer subset)
- ☐ Local stats dashboard (tokens saved, cost avoided, streaks)
- ☐ High-contrast theme + accessibility audit

Phase 4: Launch & Enterprise (Weeks 25–32)

Public launch, community building, and enterprise pipeline.

- ☐ Launch on GitHub + r/LocalLLaMA + Hacker News
- ☐ Discord community setup
- ☐ YouTube teardown series (first 3 episodes)
- ☐ Enterprise on-prem packaging (Docker + vLLM connector)
- ☐ RBAC + audit logging + policy enforcement
- ☐ SOC 2 Type II preparation
- ☐ Pro tier billing integration (Stripe)

- ☐ Plugin registry launch

Appendix A: Risk Register

RISK	LIKELIHOOD	IMPACT	MITIGATION
7B model quality insufficient for reliable edits	Medium	High	Fallback to cloud BYOK; monitor model releases (Qwen 3, Llama 4) for quality jumps
Scope creep delays MVP	High	Critical	Ruthless Phase 1 scope — ship editor + completions + chat only; defer MCP, Whisper, Canvas
Electron-free Tauri limits Monaco integration depth	Medium	Medium	Prototype Monaco-in-Tauri WebView early; identify blockers in Week 1
MCP tool calling unreliable with local models	High	Medium	Curated tool list with per-model ratings; retry logic; don't promise "1-click" until validated
Apple Silicon MLX ecosystem immature	Low	Low	llama.cpp Metal is the primary backend; MLX is optional acceleration
Enterprise sales cycle too long for startup runway	Medium	High	Focus on Pro tier (\$15–20) for initial revenue; enterprise is gravy, not the lifeline

Open Studio Blueprint v2.0 — Revised August 2026